

Algorithmes et Pensée Computationnelle

Structure de données, Itération et récursivité - Basiques

Le but de cette séance est de mettre en pratique les connaissances du cours sur les structures de données (tuples, listes, dictionnaires), l'itération et la récursivité. Au terme de cette séance, l'étudiant saura distinguer structures immuables et non immuables, et écrire des programmes utilisant itération et récursivité.

Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier "Exercices > Code". Le temps mentionné (🕒) est à titre indicatif.

1 Structures de données

1.1 Tuples

Pour rappel, les tuples sont des listes d'éléments immuables, ce qui signifie que ces listes ne peuvent pas être modifiées. L'une des particularités des tuples est la possibilité d'y inclure des données de types différents.

Les tuples sont utiles pour stocker des données que l'on va réutiliser plus tard.

En Python, pour créer un tuple, il suffit de définir une variable et de lui assigner des valeurs entre parenthèses et séparer les valeurs entre elles par des virgules :

>_ Exemple

```
1 mon_tuple = (1, "une chaîne de caractères", 3, 4)
```

Question 1: (🕒 5 minutes) [Python] Manipulation d'un tuple

Créez un tuple nommé `mon_tuple` contenant les chiffres 1,2,3,4 et 5. Affichez le 4ème élément de ce tuple. Obtenez le nombre d'éléments contenus dans votre tuple et stockez le résultat dans une nouvelle variable nommée `taille_tuple`. Pour finir, affichez le contenu de `taille_tuple`.

Question 2: (🕒 5 minutes) [Python] Manipulation d'un tuple

Qu'affiche le code suivant ?

```
1 mon_tuple = (1, 2, 3, 4, 5, 6)
2
3 if 6 in mon_tuple :
4     print("6 est contenu dans le tuple")
5 else :
6     print("6 n'est pas contenu dans le tuple")
```

1.2 Listes

La différence entre une liste et un tuple (en Python) est que l'on peut modifier les éléments d'une liste. Par exemple, il est possible de remplacer un élément d'une liste par un autre tandis qu'il est impossible de le faire avec un tuple.

Pour créer une liste, il suffit de définir une variable avec des valeurs entre crochets :

>_ Exemple

```
1 ma_liste = [1, 2, 3, 4, 5]
```

Question 3: (🕒 5 minutes) [Python] Manipulation d'une liste - 1

Créez une liste nommée `ma_liste` contenant les nombres 1,2,3,4 et 5. Stockez le deuxième élément de la liste dans une variable nommée `element_2`, puis stockez la taille de la liste dans une variable nommée `taille_liste`. Affichez ces deux variables.

Question 4: (🕒 5 minutes) [Python] Manipulation d'une liste - 2

Qu'affiche le code suivant ?

```
1 ma_liste = [1, 2, 2, 4, 5]
2 ma_liste[2] = 3
3 ma_liste.append(6)
4 ma_liste.insert(0, 0)
5 print(ma_liste)
```

Choisissez parmi les éléments suivants :

- [1, 2, 2, 4, 5, 6]
- [0, 1, 2, 3, 4, 5]
- [0, 1, 2, 3, 4, 5, 6]
- [0, 1, 3, 2, 4, 5, 6]

1.3 Dictionnaires

Les dictionnaires sont des listes associatives, c'est-à-dire des listes qui lient une valeur à une autre. Dans un dictionnaire en papier, les mots sont liés à leur définition.

Dans un dictionnaire Python, les éléments sont liés par une relation dite `clé`, `valeur`. La clé étant le moyen de "retrouver" notre valeur dans notre dictionnaire. Par exemple, dans un dictionnaire en papier, nous pouvons retrouver une définition en cherchant le mot qui lui correspond, alternativement, nous pouvons retrouver le contenu d'une page d'un livre en utilisant le numéro de celle-ci, dans ce cas le numéro est la clé et le texte de la page, la valeur.

Pour créer un dictionnaire, il suffit de lister les couples `clé`, `valeurs` entre deux accolades :

```
1 elon_musk = {
2     "prénom": "Elon",
3     "nom": "Musk",
4     "age": 48,
5     "talents": ["programmation", "entrepreneuriat", "aéronautique"]
6 }
7
```

Question 5: (🕒 5 minutes) [Python] Manipulation d'un dictionnaire - 1

Créez un dictionnaire nommé `fr_eng` contenant les associations suivantes :

```
1 {
2     "chat": "cat",
3     "chien": "dog",
4     "oiseau": "bird",
5     "poule": "chicken",
6     "papillon": "butterfly",
7     "souris": "mouse",
8     "ours": "bear",
9     "mouton": "sheep",
10    "cochon": "pig"
11 }
```

Ce dictionnaire contient des mots en français (clés) associés à leur traduction en anglais (valeurs).

Accédez à la traduction du mot “souris”, et stockez le résultat dans une variable nommée `souris_traduite`, puis calculez le nombre d’éléments contenus dans le dictionnaire et stockez le résultat dans une variable nommée `taille_fr_eng`. Affichez le contenu des deux variables que vous venez de créer.

Question 6: (🕒 5 minutes) [Python] **Manipulation d’un dictionnaire - 2**

Qu’affiche le programme suivant ?

```
1 elon_musk = {
2     "prénom": "Elon",
3     "nom": "Musk",
4     "age": 48,
5     "talents": ["programmation", "entrepreneuriat", "aéronautique"]
6 }
7
8 print(elon_musk["talents"][2])
9 print("aéronautique" in elon_musk["talents"])
```

Choisissez parmi les possibilités suivantes :

- (A) programmation
True
- (B) Musk
False
- (C) 48
False
- (D) aéronautique
True

Question 7: (🕒 5 minutes) [Python] **Manipulation d’un dictionnaire - 3**

1. Vous disposez de deux dictionnaires contenant les notes d’étudiants dans différentes matières : `notes_biologie` et `notes_chimie`. Chaque dictionnaire associe le nom d’un étudiant à sa note dans la matière correspondante. Votre objectif est de créer un nouveau dictionnaire qui regroupe les notes de chaque étudiant sous forme de liste, où chaque liste contient d’abord la note de biologie puis la note de chimie. Complétez le programme suivant :

```
1 if __name__ == '__main__':
2     notes_biologie = {"Charlie": 2, "Alice": 6, "Bob": 5.5}
3     notes_chimie = {"Charlie": 4, "Alice": 5, "Bob": 6}
4     # Votre code ici:
5
6     # Résultat:
7     # {"Charlie": [2, 4], "Alice": [6, 5], "Bob": [5.5, 6]}
```

2. Affichez le nombre d’étudiants dont la note moyenne est plus grand que 5.

1.4 Structures de données en Java

Dans cette partie, les exercices devront être effectués en Java.

Les syntaxes des principales structures de données abordées dans cette partie sont les suivantes :

Déclaration et affectation d’un tuple (array) en Java de type `Object[]` :

```
1 Object[] mon_tuple = {1, 2, 3, 4};
```

Déclaration et affectation d’un tuple (array) en Java de type `String[]` :

```
1 String[] mon_tuple = {"UNIL", "ACT"};
```

Pour la déclaration généralement, on écrit : **T[] nom_variable**, où T est le type d'éléments dans le tuple.

Déclaration et affectation d'une liste en Java :

```
1 List ma_liste = List.of(1,2,3,4);
```

Déclaration et affectation d'un dictionnaire en Java :

```
1 HashMap mon_dictionnaire = new HashMap(Map.of("un", 1, "deux", 2));
```

> Informations utiles

— En Java, les listes sont immuables, comme les tuples en Python. Au cas où vous devriez modifier une liste, on vous recommande d'utiliser une liste liée (LinkedList). Une liste liée se déclare comme suit :

```
1 List liste = List.of(1,2,3,4);  
2 LinkedList ma_liste = new LinkedList(liste);
```

— En Java, les listes ne peuvent contenir que des éléments de même type.

Question 8: (🕒 10 minutes) [Java] Manipulation des listes

Créez une liste nommée `ma_liste` contenant les nombres 1,2,3,4 et 5. Affichez le deuxième élément de la liste ainsi que la taille de la liste.

Créez une liste `ma_liste_m` liée à la liste `ma_liste`. Ajoutez le chiffre 6 à la fin de la liste, et le chiffre 0 au début de cette dernière.

Ajoutez ceci au début de votre code :

```
1 import java.util.List;  
2 import java.util.LinkedList;
```

Ajoutez ceci à la fin de votre code :

```
1 for(int i=0;i<ma_liste_m.size();i++){  
2     System.out.println(ma_liste_m.get(i));  
3 }
```

Question 9: (🕒 15 minutes) [Java] Manipulation de dictionnaires

Créez un dictionnaire `mon_dictionnaire` contenant les associations suivantes :

```
("étudiants", 14000, "enseignants", 2300, "collaborateurs", 0)
```

Affichez le nombre d'étudiants. Obtenez la taille du dictionnaire et stockez le résultat dans une variable `taille_dictionnaire`. Affichez le contenu de `taille_dictionnaire`. Modifiez la valeur de `collaborateurs`, remplacez 0 par 950. Rajoutez un nouvel élément `pays` dans votre dictionnaire avec pour valeur 86.

Ajoutez ceci au début de votre code :

```
1 import java.util.HashMap;  
2 import java.util.Map;
```

Ajoutez ceci à la fin de votre code :

```
1 for (String key : mon_dictionnaire.keySet()) {  
2     System.out.println(key + " : " + mon_dictionnaire.get(key));  
3 }
```

2 Itération

Quelques rappels de concepts théoriques :

- L'itération désigne l'action de répéter un processus (généralement à l'aide d'une boucle) jusqu'à ce qu'une condition particulière soit remplie.
- Il y a deux types de boucles qui sont majoritairement utilisées :

Boucle for — Python : Une boucle `for` permet d'itérer sur un ensemble. Cet ensemble peut être une liste, un dictionnaire, une collection, une chaîne de caractères, un tuple. La syntaxe d'une boucle en Python est la suivante `for nom_de_variable in variable_ensemble` où `variable_ensemble` est la variable qui stocke l'ensemble sur lequel vous voulez itérer. La variable `nom_de_variable` prendra la valeur de chaque élément dans l'ensemble, un par un.

Boucle for — Java La syntaxe en Java est la suivante : `for (type nom_de_variable; condition de fin de boucle; incrémentation à chaque itération)` suivi d'une accolade. Il est également possible d'utiliser la syntaxe suivante pour itérer directement sur les éléments d'une liste : `for (var nom_de_variable : nom_de_la_liste_ou_tuple)` ou `for (var nom_de_variable : nom_du_dictionnaire.keySet())` pour un dictionnaire. Cette notation est appelée `for each`.

Boucle while : Les boucles `while` sont des boucles qui s'exécutent de façon continue jusqu'à ce qu'une condition soit remplie. La syntaxe est la suivante : En Python, on écrit d'abord `while`, suivi de la condition à atteindre.

En Java, il n'y a pas de différence majeure à part le fait qu'il faut mettre entre parenthèses la condition et les deux points sont remplacés par des accolades.

- La fonction `range(n)` permet de créer un ensemble de nombres de 0 à la valeur passée en argument moins 1 ($n-1$). Lorsqu'elle est combinée à une boucle `for`, on peut itérer sur une liste de nombres de 0 à $n-1$.
- Si vous avez fait une erreur dans votre code, il se peut que la boucle `while` ne remplisse jamais la condition lui permettant de sortir. On parle dans ce cas d'une boucle infinie. Ceci peut entraîner un crash de votre programme, voire même de votre ordinateur. Il faut toujours s'assurer qu'il y ait une condition valide dans une boucle `while`.

Question 10: (🕒 5 minutes) [Python] Boucle for

Qu'affiche le code suivant ?

```
1 ma_liste = [1,2,3,4,5]
2 for c in ma_liste :
3     print(c)
4
```

Choisissez parmi les possibilités suivantes :

(A)	(B)	(C)	(D)
1	5	3	4
2	4	2	3
3	3	1	5
4	2	5	1
5	1	4	2

Question 11: (🕒 5 minutes) [Java] Boucle for

Qu'affiche le code suivant ?

```

1 List ma_liste = List.of(2,3,4,5,6);
2 for(int i=0;i<ma_liste.size();i++){
3     System.out.print(ma_liste.get(i));
4 }
5

```

Choisissez parmi les possibilités suivantes :

- (A) 5 4 3 2 1
- (B) 1 2 3 4 5
- (C) 4 3 5 1 2
- (D) 2 3 4 5 6

Question 12: (🕒 5 minutes) [Python] **Boucle while**
Qu'affiche le code suivant ?

```

1 i = 0
2 result = ""
3 while i<5:
4     result = result + str(i)
5     i += 1
6 print(result)
7

```

Choisissez parmi les possibilités suivantes :

- (A) 0 1 2 3 4
- (B) 1 2 3 4 5
- (C) 4 3 2 1 5
- (D) 2 3 4 0 1

Question 13: (🕒 5 minutes) [Java] **Boucle while**
Qu'affiche le code suivant ?

```

1 int i = 0;
2 while(i<5){
3     i++;
4     System.out.print(i + " ");
5 }
6

```

Choisissez parmi les possibilités suivantes :

- (A) 0 1 2 3 4
- (B) 1 2 3 4 5
- (C) 4 3 2 1 0
- (D) 5 4 3 2 1

Question 14: (🕒 5 minutes) [Python] **Boucle for et fonction range()** - 1
Qu'affiche le code suivant ?

```

1 ma_liste = ["J'aime", "Python"]
2 for x in range(len(ma_liste)):
3     print(x)
4     print(ma_liste[x])

```

Choisissez parmi les possibilités suivantes :

(A)	(B)	(C)	(D)
0	J'aime	J'aime	1
J'aime	1	0	J'aime
1	Python	Python	2
Python	2	1	Python

Question 15: (🕒 5 minutes) [Python] **Boucle for et fonction range() - 2**

En utilisant la fonction `range()` et une boucle `for`, calculez la somme des entiers de 0 à 20 et affichez le résultat.

Question 16: (🕒 10 minutes) [Python] **Boucle while et input()**

À l'aide d'une boucle `while`, demandez à l'utilisateur de rentrer une valeur. Tant que cette valeur ne correspond pas à 10, le programme redemande à l'utilisateur une nouvelle valeur.

Question 17: (🕒 5 minutes) [Python] **Listes en compréhension**

Définissez une liste `nombres` contenant l'ensemble des nombres compris entre 0 et 10 (bornes incluses). Ensuite, créez une liste en compréhension qui parcourt `nombres`, récupère tous les nombres pairs et les stocke dans une nouvelle variable de type `list` nommée `nombres_pairs`. Affichez le contenu de la variable `nombres_pairs`.

Question 18: (🕒 10 minutes) [Java] **Itérateurs**

En Java, définissez une liste `nombres` contenant l'ensemble des nombres compris entre 0 et 10. En utilisant un `Iterator`, supprimez tous les nombres impairs présents dans `nombres`. Affichez le contenu de `nombres`.

3 Récursivité

La récursivité est le fait d'appeler une fonction de façon récursive c'est-à-dire une fonction qui s'appelle elle-même de façon directe ou indirecte.

Question 19: (🕒 5 minutes) [Python] **Lecture de code - 1**

Qu'affiche le programme suivant ?

```
1 def fonction_p(x) :  
2     print(x)  
3     if x == 0 :  
4         pass  
5     else :  
6         fonction_p(x-1)  
7  
8 fonction_p(5)
```

Choisissez parmi les possibilités suivantes (On écrit tout à la même ligne par souci de concision) :

- 4 3 2 1 0
- 5 4 3 2 1 0
- 0 1 2 3 4
- 1 2 3 4 5 0

Question 20: (🕒 5 minutes) [Python] **Lecture de code - 2**

Qu'affiche le programme suivant ?

```

1 def fonction_p(x) :
2     if x == 0 :
3         pass
4     else :
5         fonction_p(x-1)
6         print(x)
7
8 fonction_p(5)

```

Choisissez parmi les possibilités suivantes (On écrit tout à la même ligne par souci de concision) :

- 4 3 2 1 0
- 5 4 3 2 1
- 0 1 2 3 4
- 1 2 3 4 5

Question 21: (🕒 10 minutes) [Python et Java] **Fonction factoriel()**

Ecrivez la fonction `factoriel()` suivant une approche récursive.

Pour rappel, cette fonction prend un entier et retourne le factoriel de ce dernier tel que :

$\text{factoriel}(1) = 1$, $\text{factoriel}(2) = 1 * 2 = 2$, $\text{factoriel}(3) = 1 * 2 * 3 = 6$, $\text{factoriel}(4) = 1 * 2 * 3 * 4 = 24, \dots$

Question 22: (🕒 10 minutes) [Java] **Méthode getBinaryRepresentation()**

Le calcul de la représentation binaire d'un entier plus grand que 0 peut être résolu de manière récursive :

$$f(a) = \begin{cases} f\left(\frac{a}{2}\right) + "0" & \text{si } a \bmod 2 = 0 \\ f\left(\frac{a}{2}\right) + "1" & \text{si } a \bmod 2 = 1 \end{cases} \quad (1)$$

Le cas de base étant :

$$f(0) = "" \text{ (la chaîne vide)} \quad (2)$$

Dans cette formule, $f(a)$ renvoie la représentation binaire sous forme d'une chaîne de caractères du nombre entier $a \geq 1$ et le symbole $+$ fait une concaténation de chaînes de caractères.

Ecrivez la méthode `getBinaryRepresentation(String a)` équivalente à f (à part le fait que l'entrée soit une chaîne de caractères) suivant une approche récursive.